

EECS 498 · AASE · FALL 2026

# Modern agentic CLIs

LECTURE 05 · SEP 15, 2026

## Lab01 left one question open

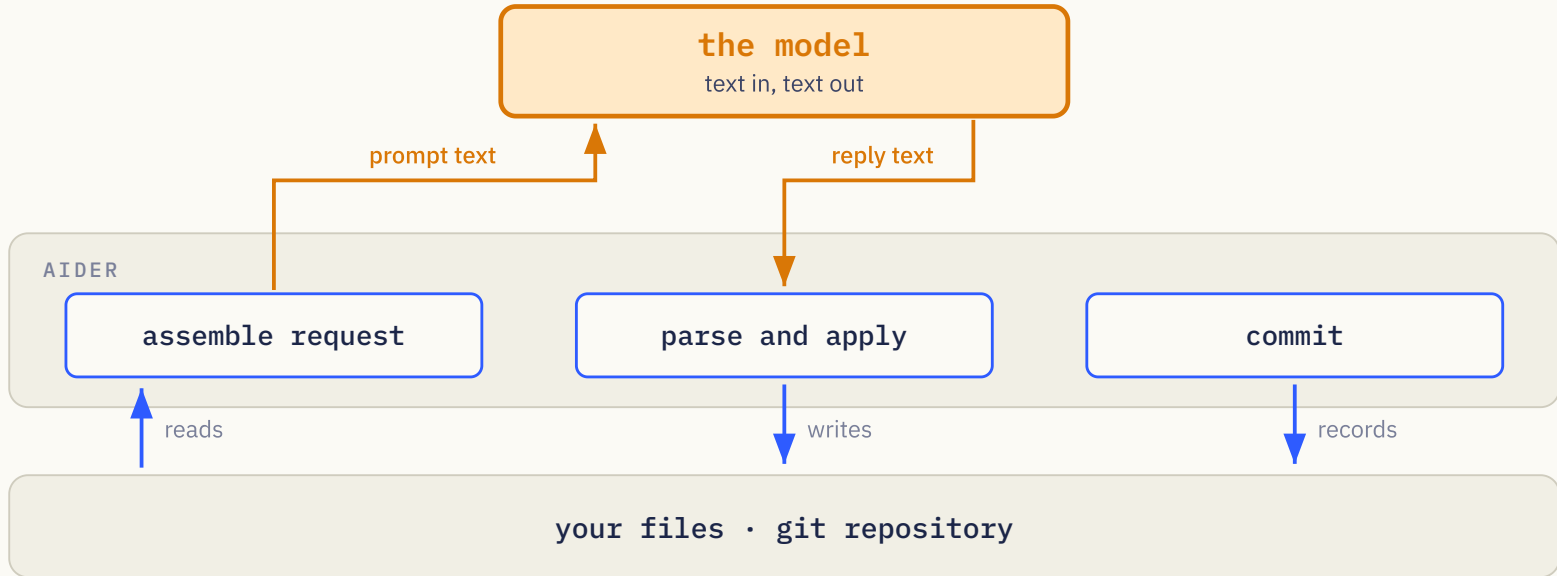
You sorted the prompt by who chose each part.

| Part of the prompt        | Chosen by                    |
|---------------------------|------------------------------|
| System prompt             | Aider, by mode               |
| Added and read-only files | You, by command              |
| Repo map                  | Aider, by a ranking          |
| History                   | You start it, Aider trims it |
| Your message              | You                          |

Two rows you never touched. One of them passed for intelligence.

AIDER IS ONE

# Aider, the model, and the repo



The model touches no file.

THE LOOP, WRITTEN OUT

# Six steps, in order

## 1 assemble

you and Aider,  
per the table

## 2 answer

the model. Text  
out

## 3 parse

Aider, by a fixed  
rule

## 4 apply

Aider, or it  
refuses

## 5 repair

only if 4 failed

## 6 commit

Aider, every  
time

One request, start to finish, and the order never varies. Step 2 is the only one the model takes.

# Repair is an insertion, not a stage

- A block that does not match is **normal**, not exceptional
- Aider writes the failure back as a prompt, and step 2 runs again
- Up to three times, then it gives up and tells you

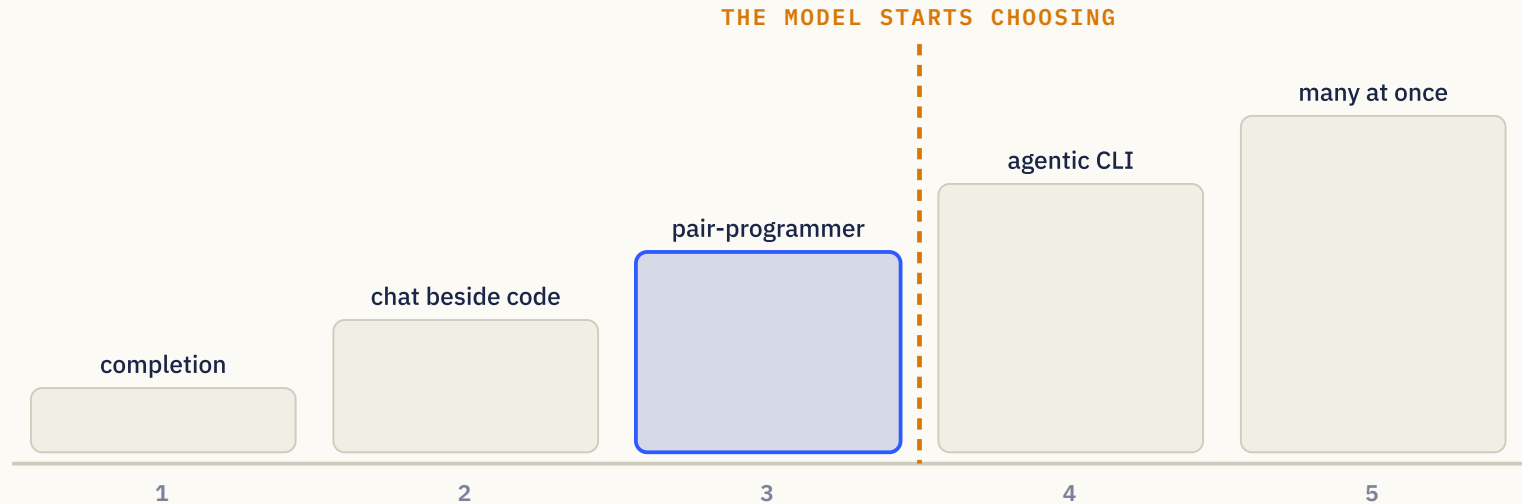
It re-enters at step 2. It never chooses a different step, and it never chooses a different file.

Who picks the **next action**?

# The autonomy gradient

FIVE RUNGS, AND WHERE YOU HAVE BEEN STANDING

# Five rungs, not a ranking



The blue rung is where you have spent two weeks. M1 is this rung; M2 and M3 are rung 4.

## You are on rung 3

### At rung 3

|               |                                     |
|---------------|-------------------------------------|
| You hand over | One bounded edit task               |
| You choose    | The files, and where the task ends  |
| It automates  | Matching, repair, tests, the commit |
| You check     | Between tasks                       |

Real automation inside a boundary you drew.

## Higher rungs spend more to ask less



### Cost per unit of work

A run that searches, reads five files and runs your tests three times spends far more attention than one bounded edit.



### Decisions you make yourself

Fewer choices per finished outcome, and less of your attention spent on each one.

## Rung 3 fits a change you can name

A two-line fix in a file you can point at is cheaper and safer at rung 3.

Paying rung 4 prices for it buys nothing.

**The skill is picking the rung, and you cannot pick what you cannot see.**

# What makes a tool agentic

FOUR PRIMITIVES, AND A TEST YOU CAN APPLY

## Four primitives, not a vibe



### Loop

Takes another step  
without being asked.



### Tools

Actions beyond  
emitting text.



### Planning

Breaks a goal into  
steps.



### Memory

Something survives the  
turn.

"Agentic" is used to mean "recent." These four are checkable.

## Aider has all four

| Primitive | Where it already is in Aider                               |
|-----------|------------------------------------------------------------|
| Loop      | A failed edit becomes the next prompt; tests can feed back |
| Tools     | Reads, writes, commits, runs the commands you configured   |
| Planning  | A plan drafted in prose, stopping for a human to read      |
| Memory    | Session history, plus a repo map of code you never added   |

The difference is **who chooses.**

THE MOVE

# One choice changes the rung

At rung 3 you choose the files, the boundary, and whether to run the plan.

Move the choosing of the next action from the harness to the model, and everything people find surprising about agentic tools follows.

One change. Not a new category of intelligence.

# Chat break

FIVE MINUTES, ON THE TOOL YOU ALREADY KNOW

## Give one choice away

Two weeks of Aider on taskr. Three choices are still yours every time: which files it sees, where the task ends, whether the plan runs.

Hand **exactly one** of them to the model.

**Then say what you would have to build before you would let that loose on a repository you care about.** Which is more work, the handover or the guardrail?

# PollEv

<https://PollEv.com/marcusdarden365>

# The same request, one rung up

LIVE, ON THIS MACHINE

## You know what this took at rung 3

Add a `--priority` flag to taskr's add command.

You added three files, because a file you do not add is not in the context. You framed the task. You read the diff. You found the missing write yourself.

# Nobody types /add

## It searches

Finds where arguments are parsed, without being told.

## It reads

Follows the code to the files the change actually touches.

## It runs

Executes the tests and reacts to what they say.

Context assembly became something the program does by acting.

## Each result chooses the next step

```
search "add_argument" → three hits, one in cli.py
read cli.py           → finds the parser
read store.py         → finds the save path
edit cli.py, task.py, store.py
run tests             → 1 failed
edit store.py
run tests             → passed
```

Nothing in that order was written down in advance, by anyone.

HONESTY

# Today's run is a fact about today

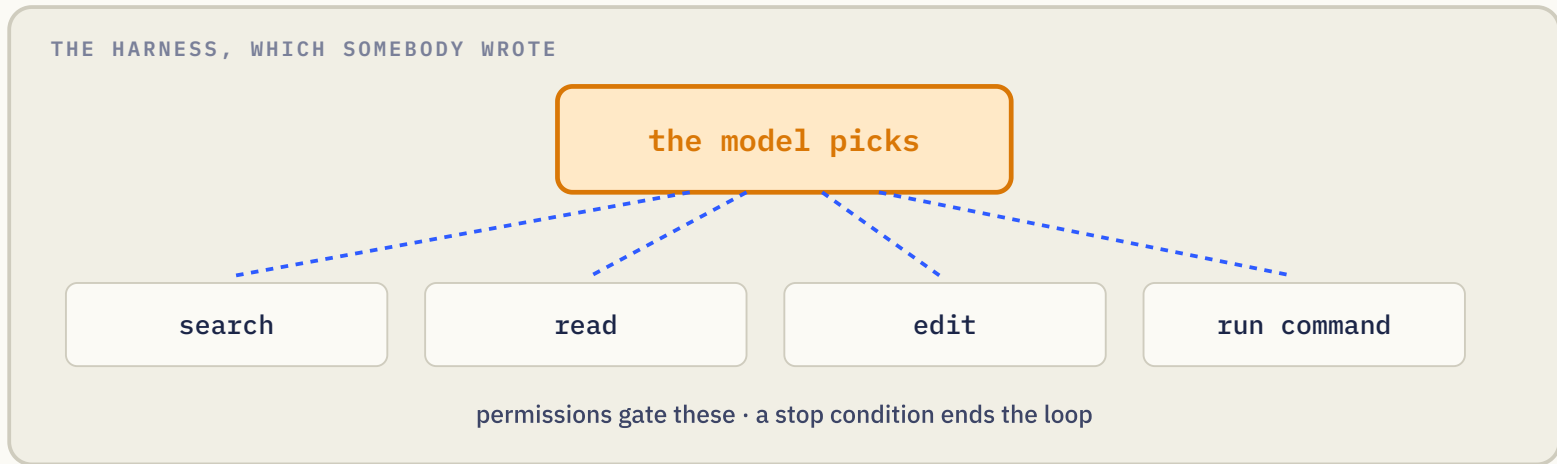
What you just watched is what this model and this build did with this repository, this afternoon.

**A run that goes badly is more useful than one that goes well. Do not rescue it too quickly.**

# What the harness still decides

THE PART THE DEMO HIDES

# The model picks from a menu it did not write



Somebody decided a search tool exists and a database client does not.

## Permissions are the real safety layer

You were handed this shape in lab. Nothing changes on disk until a human approves it, and that holds whatever the model says, because the program enforces it, not the prompt.

**The same question scales up: when the model can run commands, which ones run without being asked?**

Prose asks. Code and permissions  
**enforce.**

## STOPPING

# A loop needs a reason to stop

A goal it can recognize as met. A budget it can spend. A human who interrupts.

At rung 3 the task boundary was the stop condition, and you drew it by hand every single time.

YOUR BUILD

## Rung 3 is the point, not a limitation

Exact matching. One approval before any write. One commit per accepted edit set. A guarded undo.

Those are what make the state after a failure something you can say in a sentence and assert in a test.

Build that, and you have built what a loop needs underneath it.

# The trace is the evidence

When a run produces the wrong outcome, the output does not tell you why.

Which step picked the wrong file? Which result did it misread? Which missing tool did it improvise around?

**"The model is bad at this" is the answer people reach for, and rarely the one the evidence supports.**

# The build is due in three weeks

| When                | What                             |
|---------------------|----------------------------------|
| Thu Sep 17          | L06 closes Apply                 |
| Mon 21 / Tue 22     | Lab02, supported build time      |
| Thu Sep 24, evening | Hackathon 1, on your own project |
| Tue Oct 6, 11:59 PM | The build is due                 |

TAKE THESE WITH YOU

# Three questions

1. Where on the gradient is the tool you reach for by default, and did you choose that or inherit it?
2. What would have to be true before you would let a model run a command without asking you?
3. What would you need to see in a trace to be convinced a run failed for a reason other than the model?

Thursday, the loop stops being  
**someone else's.**

# Questions?